

Redis + Celery + NLP Backend Setup Guide For Team

Objective

This document explains how every team member can set up the backend project exactly like the working system.

This setup includes:

- Python environment
 - Backend dependencies
 - Redis installation
 - Celery worker setup
 - FastAPI backend setup
 - NLP model integration
 - Database connection
 - API testing workflow
 - Common issues and fixes
-

1. Backend Architecture Overview

Current Backend Structure

```
backend/
|
├── .env
├── auth_utils.py
├── database.py
├── dbmodels.py
├── main.py
├── models.py
├── requirements.txt
|
├── core/
│   └── celery_app.py
|
├── models/
│   ├── model.pkl
│   └── vectorizer.pkl
|
├── routers/
│   ├── admin.py
│   └── bookings.py
```

```
|   |— intent.py
|   |— unified_auth.py
|   |— userinput.py
|   |— users.py
|   |— workers.py
|
|— services/
|   |— entity_extractor.py
|   |— nlp_service.py
|   |— text_normalizer.py
|
|— tasks/
|   |— celery_tasks.py
```

2. End-to-End Workflow

Actual System Flow

```
Frontend
↓
POST API Request
↓
FastAPI Router
(userinput.py)
↓
Celery Background Task
(tasks/celery_tasks.py)
↓
Redis Queue
↓
Celery Worker Picks Task
↓
NLP Service
(nlp_service.py)
↓
Text Normalization
(text_normalizer.py)
↓
Entity Extraction
(entity_extractor.py)
↓
ML Model Prediction
(model.pkl + vectorizer.pkl)
↓
Intent Prediction
↓
Database Save
```

↓
GET Intent API
(intent.py)
↓
Frontend Response

3. What Redis Is Doing

Redis is acting as:

- Message broker
- Queue system
- Communication layer between FastAPI and Celery

Without Redis:

- Celery cannot receive tasks
- Background processing fails
- NLP async execution will not work

4. What Celery Is Doing

Celery handles:

- Background NLP processing
- Async task execution
- Heavy processing outside API thread

Instead of:

User → Wait for NLP → Response

Celery changes it to:

User → Task queued → Worker processes → Response stored

This improves:

- Performance
 - Scalability
 - Reliability
 - Non-blocking APIs
-

5. System Requirements

Every team member should install:

Required Software

Software	Version
Python	3.10+
Redis	Latest Stable
Git	Latest
VS Code	Recommended
Postman	Recommended

6. Step-by-Step Installation

STEP 1 — Install Python

Download Python:

- Official site: <https://www.python.org/downloads/>

IMPORTANT:

During installation enable:

✓ Add Python to PATH

Verify:

```
python --version
```

Expected:

```
Python 3.10.x
```

STEP 2 — Install Git

Download:

- <https://git-scm.com/downloads>

Verify:

```
git --version
```

STEP 3 — Clone Project

```
git clone <YOUR_GITHUB_REPO_URL>
```

Open project:

```
cd backend
```

STEP 4 — Create Virtual Environment

Inside backend folder:

```
python -m venv venv
```

Activate:

Windows

```
venv\Scripts\activate
```

Mac/Linux

```
source venv/bin/activate
```

If activated correctly:

```
(venv)
```

appears in terminal.

STEP 5 — Install Backend Requirements

```
pip install -r requirements.txt
```

This installs:

- FastAPI
- Uvicorn
- SQLAlchemy
- Celery
- Redis
- Scikit-learn
- NLP dependencies

Verify:

```
pip list
```

STEP 6 — Install Redis

Windows Method (Recommended)

Option 1 — Docker (Most Reliable)

Install Docker Desktop:

<https://www.docker.com/products/docker-desktop/>

Run Redis container:

```
docker run -d -p 6379:6379 redis
```

Check running containers:

```
docker ps
```

Expected:

```
redis
```

Option 2 — Native Redis for Windows

Download:

<https://github.com/tporadowski/redis/releases>

Install Redis.

Start Redis server.

Verify:

```
redis-cli ping
```

Expected:

```
PONG
```

Mac Installation

```
brew install redis  
brew services start redis
```

Verify:

```
redis-cli ping
```

Linux Installation

```
sudo apt update  
sudo apt install redis-server
```

Start:

```
sudo service redis-server start
```

Verify:

```
redis-cli ping
```

Expected:

```
PONG
```

7. Environment Variables (.env)

Current backend contains:

```
DATABASE_URL=  
SECRET_KEY=  
REDIS_URL=redis://localhost:6379/0
```

IMPORTANT:

Every teammate must update:

- Database credentials
- Secret key
- Environment-specific values

Do NOT hardcode production values.

8. Celery Configuration

File

```
backend/core/celery_app.py
```

Purpose:

- Connect Celery with Redis
- Configure background workers

Typical configuration:

```
from celery import Celery

celery = Celery(
    "worker",
    broker="redis://localhost:6379/0",
    backend="redis://localhost:6379/0"
)
```

What this means:

Component	Purpose
broker	Sends tasks to queue
backend	Stores task results

9. NLP Service Explanation

File

```
backend/services/nlp_service.py
```

This is the main NLP pipeline.

Responsibilities:

- Normalize text
- Extract entities
- Predict intent
- Return confidence score

10. Text Normalization

File

```
services/text_normalizer.py
```

Purpose:

Converts messy user text into standardized format.

Example:

```
"My AC isnt wrking!!!"
```

becomes:

```
"my ac is not working"
```

Why important:

Without normalization:

- NLP accuracy drops
- Intent mismatch happens
- Spelling variations break predictions

11. Entity Extraction

File

```
services/entity_extractor.py
```

Purpose:

Extracts important entities from user queries.

Example:

```
"Need AC repair in Hyderabad"
```

Extracted entities:

```
{  
  "service": "AC repair",  
  "location": "Hyderabad"  
}
```

12. ML Model Files

Folder

```
backend/models/
```

Contains:

File	Purpose
model.pkl	Trained ML model
vectorizer.pkl	Converts text to vectors

Workflow:

```
User Text
  ↓
Vectorizer
  ↓
ML Model
  ↓
Intent Prediction
```

13. Running Redis

Before starting backend:

Ensure Redis is running.

Test:

```
redis-cli ping
```

Expected:

```
PONG
```

If Redis is not running:

Celery will fail immediately.

14. Running FastAPI Backend

Inside backend folder:

```
uvicorn main:app --reload
```

Expected:

```
Uvicorn running on http://127.0.0.1:8000
```

Swagger:

```
http://127.0.0.1:8000/docs
```

15. Running Celery Worker

Open NEW terminal.

Activate virtual environment.

Run:

```
celery -A tasks.celery_tasks worker --loglevel=info
```

Expected logs:

```
Connected to redis://localhost:6379/0
```

and:

```
ready
```

IMPORTANT:

Backend and Celery must run in separate terminals.

16. How APIs Are Working

POST Router

File:

```
routers/userinput.py
```

Purpose:

- Accept user query
 - Push task into Celery
 - Store query
-

Intent Router

File:

```
routers/intent.py
```

Purpose:

- Fetch NLP result
 - Return predicted role/intent
-

17. Current Architecture Strengths

Your architecture already has good foundations:

Good Decisions

✓ FastAPI architecture ✓ Router separation ✓ Service layer separation ✓ Background task processing ✓ Redis queue system ✓ NLP modularization ✓ ML model isolation ✓ Entity extraction layer

18. Current Weaknesses

These are real issues your team should improve.

Problem 1 — Keyword Overlap

Example:

"My cooler is not working"

Wrong prediction:

AC_Mechanic

Expected:

Electrician

Cause:

- Weak intent boundaries
- Keyword collision
- Training dataset imbalance
- Overfitting toward AC keywords

Fix:

- Add intent-specific examples
- Increase negative examples
- Improve preprocessing
- Add confidence threshold
- Add hybrid rules + ML

Problem 2 — Query ID Dependency

Current system:

POST → returns query_id
GET → manually paste query_id

This is not scalable.

Better flow:

POST user query
↓
Celery processing

↓
Automatically save intent
↓
Frontend fetches latest result

Your next improvement should remove manual query_id copying.

19. Recommended Production Improvements

Priority Improvements

Improvement	Priority
Async DB handling	High
Dockerization	High
Central logging	High
Retry handling	High
Better NLP dataset	Critical
Intent confidence thresholds	Critical
API authentication	High
Rate limiting	Medium
Monitoring dashboard	Medium

20. Recommended Startup Order

Every teammate should follow THIS exact order.

Terminal 1

Start Redis.

Terminal 2

Start FastAPI:

```
uvicorn main:app --reload
```

Terminal 3

Start Celery:

```
celery -A tasks.celery_tasks worker --loglevel=info
```

21. How To Verify Entire System

Step 1

Open Swagger:

```
http://127.0.0.1:8000/docs
```

Step 2

Test POST endpoint.

Example:

```
{  
  "query": "My AC is not cooling"  
}
```

Step 3

Check Celery logs.

You should see task processing.

Step 4

Check Redis logs.

Ensure queue receives tasks.

Step 5

Test GET intent endpoint.

Verify correct role prediction.

22. Common Errors and Fixes

Error

Error 10061 Connection Refused

Cause:

Redis not running.

Fix:

Start Redis.

Error

ModuleNotFoundError

Cause:

Virtual environment not activated.

Fix:

Activate venv.

Error

Cannot connect to Redis

Cause:

Wrong Redis URL.

Fix:

Check:

```
REDIS_URL=redis://localhost:6379/0
```

Error

```
Celery task not registered
```

Cause:

Wrong task import.

Fix:

Ensure:

```
from tasks.celery_tasks import process_nlp
```

is imported correctly.

23. Recommended Team Workflow

Backend Team

Responsible for:

- APIs
 - Database
 - Celery
 - Redis
 - Authentication
-

NLP Team

Responsible for:

- Training data
- Intent mapping

- Entity extraction
 - Confidence tuning
 - Misclassification reduction
-

Frontend Team

Responsible for:

- Query submission
 - Polling results
 - Displaying intent
 - Error handling
-

24. Recommended Next Phase

Your current architecture is Phase 1.

Next phase should include:

Better NLP

Move from:

Keyword-heavy ML

toward:

Transformer-based NLP

Examples:

- BERT
 - DistilBERT
 - Sentence Transformers
 - Intent embeddings
-

Better Queue Handling

Add:

- Retry mechanism
- Dead-letter queue

- Task status tracking
- Async result backend

Better Deployment

Use:

- Docker
- Docker Compose
- Nginx
- Gunicorn
- Cloud deployment

25. Recommended Production Stack

Layer	Recommended
API	FastAPI
Queue	Redis
Worker	Celery
NLP	Transformers
DB	PostgreSQL
Deployment	Docker
Reverse Proxy	Nginx
Cloud	AWS / GCP / Azure

26. Final Setup Checklist

Before saying setup completed:

Verify All

Python

```
python --version
```

Virtual Environment

```
(venv)
```

visible.

Dependencies

```
pip list
```

Redis

```
redis-cli ping
```

returns:

```
PONG
```

FastAPI

```
http://127.0.0.1:8000/docs
```

opens correctly.

Celery

Worker connected successfully.

NLP

Intent prediction working.

27. Important Reality Check

Your current system is not yet production-grade.

It is a good learning architecture and a strong internship-level implementation.

But:

- Intent handling still fragile
- Manual query flow exists
- Dataset quality likely weak
- No monitoring
- No scaling strategy yet
- No retry/failure recovery
- No container orchestration

Do not fool yourselves into thinking this is enterprise-ready yet.

But the foundation is absolutely usable for learning, demos, internship projects, and incremental scaling.

28. Recommended Immediate Action Items

Team Should Do Next

1. Remove manual query_id flow

Highest priority.

2. Improve intent dataset

Especially confusing service categories.

3. Dockerize backend

Everyone should run same environment.

4. Add logging

Track:

- Failed predictions
 - Wrong intents
 - Celery failures
 - Redis downtime
-

5. Add confidence threshold

Low confidence should return:

```
unknown_intent
```

instead of wrong role.

29. One-Line Startup Commands

Redis

```
docker run -d -p 6379:6379 redis
```

Backend

```
uvicorn main:app --reload
```

Celery

```
celery -A tasks.celery_tasks worker --loglevel=info
```

Setup Complete

If all three are running successfully:

✓ Redis ✓ FastAPI ✓ Celery

then the NLP background architecture is connected correctly.